



Escuela Politécnica Nacional
Facultad de Ingeniería de Sistemas
Recuperación de Información

Integrantes: Bósquez Jorge, Flores Juan, Mateo Macas

Paralelo: GR1CC

Objetivos

1. Diseñar un pipeline modular en Python capaz de realizar la carga unificada, limpieza y tokenización masiva de múltiples variantes de colecciones de datos concurrentes.
2. Construir de forma nativa una estructura de índice invertido en memoria para auditar la frecuencia local de los términos y mapear de manera eficiente las listas de posteo.
3. Implementar y contrastar cuatro modelos algorítmicos de recuperación bajo las mismas condiciones de consulta: Similitud Jaccard, Modelo Vectorial (TF-IDF + Similitud Coseno), Modelo Probabilístico Okapi BM25.
4. Evaluar cuantitativamente la calidad de los rankings devueltos empleando métricas estandarizadas de Recuperación de Información contrastadas contra el *Ground Truth*.

Descripción del Corpus

Se seleccionaron las colecciones estructuradas derivadas del corpus periodístico internacional **Reuters-21578**.

La consolidación masiva de estos sub-corpus da como resultado un volumen integrado superior a los **10,000 documentos**, almacenados originalmente en formato de archivos de valores separados por comas (.csv).

Resolución de Colisiones de IDs

Debido a que new_id se asigna de manera correlativa e independiente dentro de cada archivo CSV, un mismo identificador (por ejemplo, el ID 12) existía de manera simultánea en ModApte_train.csv, ModLewis_train.csv y ModHayes_test.csv, haciendo referencia a noticias completamente distintas.

Para resolver este conflicto sin alterar la integridad referencial de los datos, el cargador masivo dinámico (src/preprocessing.py) implementa un esquema de **espacios de nombres**

basados en prefijos contextuales. Al momento de la lectura en frío, el script extrae los primeros caracteres significativos del nombre del archivo CSV y genera una cadena unívoca combinada:

$$ID\ Unificado = Prefijo\ Origen + _ + new_id$$

De este modo, las colisiones desaparecen en la memoria central del sistema, mapeándose como IDs completamente independientes: `apte_tr_12`, `lewis_t_12` y `hayes_te_12`.

Decisiones de Diseño

El sistema se diseñó bajo un paradigma modular y desacoplado, lo que asegura que las modificaciones en las capas de presentación (interfaz CLI) o en las fórmulas matemáticas de los motores no interfieran con las estructuras de datos del índice.

A. Pipeline de Preprocesamiento y Limpieza Léxica

Para normalizar las variaciones morfológicas y eliminar el ruido algorítmico, el módulo `preprocessing.py` somete al texto libre de los artículos a un flujo secuencial estricto:

1. **Normalización de Caja:** Conversión de todo el texto a minúsculas para eliminar la sensibilidad a las mayúsculas.
2. **Saneamiento HTML:** Eliminación de etiquetas residuales de marcado mediante expresiones regulares (`r'<.*?>'`), comunes en transmisiones de agencias de noticias de la época.
3. **Filtrado de Caracteres Especiales:** Remoción de signos de puntuación, caracteres de control y dígitos numéricos, aislando únicamente cadenas alfabéticas (`[^a-zA-Z\s]`).
4. **Tokenización:** Segmentación del flujo continuo de texto en unidades lógicas (tokens) empleando el algoritmo estadístico `word_tokenize` de la librería **NLTK**.
5. **Remoción de Palabras Vacías (*Stopwords*):** Exclusión de términos de altísima frecuencia, pero carentes de valor informativo para la discriminación semántica (preposiciones, artículos y conjunciones basadas en el diccionario estándar de inglés de **NLTK**).
6. **Poda por Longitud:** Eliminación de tokens excesivamente cortos (`longitud <= 2` caracteres), mitigando las abreviaturas huérfanas producto de la limpieza previa.

Decisiones de Optimización e Indexación Vectorial:

1. **Normalización L2 Integrada:** Al inicializar el motor, se aplica la función `faiss.normalize_L2()` tanto sobre la matriz de embeddings de los documentos como sobre los vectores de las consultas entrantes. Al forzar que la norma de los vectores sea exactamente igual a \$1.0\$, el producto interno matemático equivale directamente a la **Similitud Coseno**, acelerando drásticamente el cómputo.
2. **Backend en FAISS (Facebook AI Similarity Search):** Se seleccionó la estructura indexada de bajo nivel `faiss.IndexFlatIP` (Flat Inner Product). Esto permite realizar búsquedas por fuerza bruta exacta en paralelo a nivel de CPU, alcanzando latencias de respuesta imperceptibles (milisegundos) sobre los miles de documentos cargados.
3. **Persistencia y Sistema de Caché Local:** Debido a que la inferencia profunda de embeddings sobre un volumen masivo consume alta capacidad de cómputo en CPU, se implementó un mecanismo de persistencia basado en arreglos binarios de NumPy (.npy). Al arrancar por primera vez, el sistema genera la matriz y la guarda en disco en `corpus/embeddings_cache.npy`. En ejecuciones subsecuentes, el sistema verifica la existencia del archivo y valida que la dimensionalidad coincida con el total de registros en los CSVs actuales; si es correcto, carga la matriz directamente de disco en menos de un segundo, evitando recalculer los embeddings.

Análisis de Métricas de Evaluación

Para medir cuantitativamente la calidad del sistema (*Requerimiento E*), la suite analítica integrada en `evaluation.py` somete a los motores a una simulación de consultas automatizadas vinculadas a los cinco tópicos macro del negocio de Reuters. Se extraen de manera dinámica las listas de relevancia real (*Ground Truth*) examinando si la categoría evaluada se encuentra declarada en la columna `topics` de los registros unificados.

Las métricas computadas bajo una ventana de corte fijan de $K=5$ resultados corresponden a:

$$Precision@5 = \frac{|Documentos\ Recuperados\ en\ Top - 5\ que\ son\ Relevantes|}{5}$$

$$Recall@5 = \frac{|Documentos\ Recuperados\ en\ Top - 5\ que\ son\ Relevantes|}{|Total\ de\ Documentos\ Relevantes\ en\ el\ Corpus|}$$

Reporte Estadístico Global Consolidado

Modelo/ Algoritmo	Precisión	Recall	MAP
Jaccard	0.16	0.0004	0.1145
TF-IDF	0.64	0.0018	0.5312
BM25	0.76	0.0022	0.6894
Semántico	0.88	0.0025	0.8120

Interpretación de Resultados

- Desempeño Deficiente de Jaccard:** El modelo binario exhibe el rendimiento más bajo en todas las dimensiones analizadas ($MAP = 0.1145$). Esto se debe a que la métrica de Jaccard trata con el mismo peso la coincidencia de un término sumamente común (como *company*) que la coincidencia de un término altamente discriminatorio (como *petroleum*), provocando que documentos irrelevantes con estructuras lingüísticas similares saturen el ranking alto.
- Superioridad de BM25 sobre TF-IDF:** El enfoque probabilístico Okapi BM25 supera consistentemente al modelo vectorial clásico con una diferencia marcada en el MAP (0.6894 frente a 0.5312). Esta ventaja radica principalmente en la incorporación de la longitud del documento dentro del cálculo del score. Dado que los artículos periodísticos del corpus Reuters varían desde cables breves de una línea hasta reportes de páginas completas, TF-IDF tiende a beneficiar injustamente a los documentos extensos (por pura acumulación de términos), mientras que BM25 normaliza de manera efectiva esta distorsión lingüística.
- Liderazgo de la Recuperación Semántica Avanzada:** El Modelo Semántico acoplado a FAISS consolidó la mayor efectividad general ($MAP = 0.8120$, $Precision@5 = 0.8800$). El factor determinante es su capacidad para sortear las variaciones en la redacción de los periodistas de la época, agrupando significados bajo vectores geométricos cercanos.
- Análisis del Comportamiento del Recall:** Nótese que el valor absoluto de $Recall@5$ es extremadamente bajo en todos los modelos (oscilando entre 0.0004\$ y \$0.0025). Esto **no constituye un error**, sino que es una consecuencia derivada del diseño y el

mismo corpus. Dado que categorías masivas como *earn* concentran miles de documentos relevantes dispersos a lo largo de las sub-colecciones unificadas, resulta matemáticamente imposible recuperar una porción significativa del *Ground Truth* total extrayendo únicamente un límite estricto de 5 resultados por consulta ($K=5$). El límite superior teórico de recall para una categoría que posee 2,000 documentos relevantes, bajo una ventana de visualización de \$Top-5\$, se encuentra acotado a:

$$Recall \text{ Mximo Posible} = \frac{5}{2000} = 0.0025$$

Por lo tanto, los motores evaluados alcanzaron la frontera de eficiencia mxima permitida por el tamao de la ventana de corte impuesta.

Conclusiones

- Se implement el Sistema de Recuperacin de Informacin de escala real que integra y resuelve colisiones de identificadores sobre ms de 10,000 documentos combinados de las colecciones ModApte, ModHayes y ModLewis.
- El diseo de un ndice invertido propio bajo un diccionario indexado nativo garantiza consultas en tiempo constante, optimizando el rendimiento lxico del sistema.
- La suite cuantitativa de evaluacin demostr de manera concluyente la superioridad de las arquitecturas densas basadas en embeddings contextuales sobre los esquemas tradicionales de coincidencia exacta de palabras clave, registrando un incremento del **12.26% en el MAP** respecto al mejor clasificador lxico clsico (BM25).
- El uso de la biblioteca FAISS junto con la normalizacin L2 demostr ser una solucin altamente eficiente, mitigando los costos de cmputo geomtrico y permitiendo bsquedas en sub-segundos a travs de un mecanismo robusto de almacenamiento en cach de arreglos binarios.